STATS 402 - Interdisciplinary Data Analysis

Short-Horizon Temporal Optical Flow with Physics-Informed Consistency

Milestone Report: Stage 2

Sihan Yao, Yuxuan Huang

sihan.yao@dukekunshan.edu.cn, y.huang@dukekunshan.edu.cn

1. Current Status of the Project

1.1 Pipeline and Architecture Implementation

At the current stage, we have completed the basic deep learning pipeline for short-horizon temporal optical flow estimation. Building upon the formulation proposed in Stage 1, the implementation now includes:

- Pairwise flow encoder
- Joint visual-motion encoder
- PWC-style decoder
- Self-supervised training framework
- Training pipeline for both local device and DKU Computing Cluster (DKUCC)

Figure 1 shows the full current pipeline, which so far includes all basic components except for the neural operator modules. Several trainings based on this current framework have been conducted serving as baseline formation, and to evaluate the performance of our flow estimation network on a superficial level.

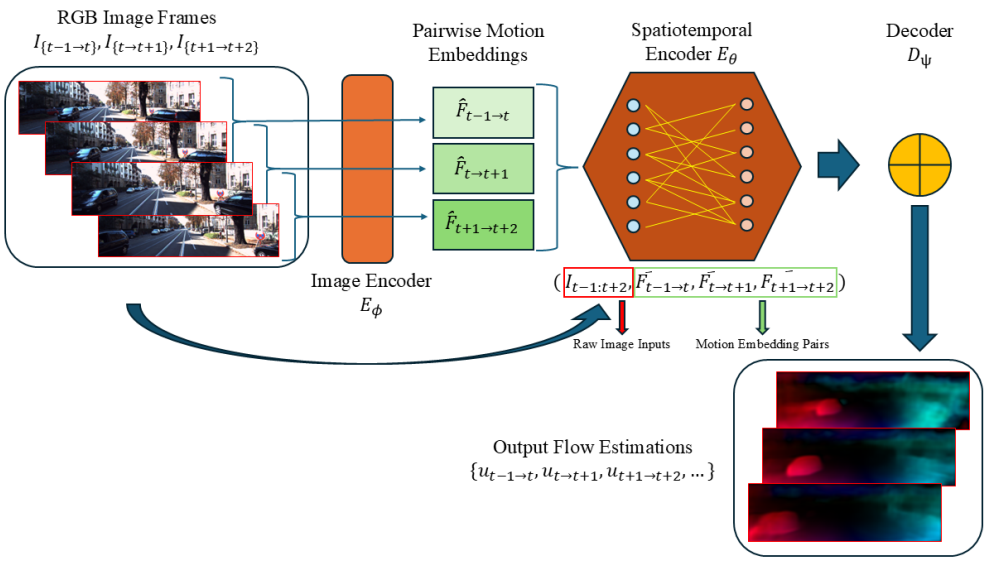


Figure 1: Current full pipeline consisting of forward encoders and decoder, without the Neural Operator module

Specifically,

(1) Pairwise Flow Encoder with Optional Sobel Enhancement

The first encoder follows a PWC-style architecture, consisting of a shared CNN backbone for feature extraction and a correlation layer for explicit motion matching (Sun et al., 2018). This segment of our pipeline follows our formulation in Milestone 1, which encodes input images frame-by-frame, and outputs one embedding for every two frames of image inputs. Based on this CNN method, we further enhance feature extraction and edge awareness with an optional Sobel-based edge augmentation drawing ideas from the guest speaker lecture in Week 3 (Zhang et al., 2009). Practically, we concatenate the image with its Sobel gradient,

$$I' = \text{concat}(I, \nabla I)$$

where the Sobel gradient magnitude is computed as:

$$\nabla I = \sqrt{(G_x * I)^2 + \left(G_y * I\right)^2}$$

If adopting this method during model initialization, the input channel would be increased from 3 (RGB) to 4 (RGB + Sobel channel). Nevertheless, as this method serves a more powerful, augmented inductive bias, it challenges model capacity and is less computationally efficient. Therefore, we have made this functionality optional.

(2) Joint Visual–Motion Encoder

The second encoder takes in both raw images and encoded vision embeddings. It adopts a two-branch CNN design: a vision branch processing raw RGB frames; and a motion branch processing pairwise embeddings. These are fused via time-aware frame-stacked 2D convolution:

$$z_t = \mathcal{F}\big(f_{\text{vision}}(I_t),\ f_{\text{motion}}(F_t)\big)$$

As we input $t + 1$ images and t flows, a mismatch between input frames occurs. We resolve this issue by passing the first t images in the forward function, which also provides an implicit time index encoding. This design avoids attention while maintaining temporal modeling efficiency. It is worth noting that this design is temporary, as we have not implemented the operator module. In further development, certain aspects of this encoder would be transferred to the neural operator, especially functions regarding time series encoding, which intends to provide the operator module with more direct input data, thereby saving computational efficiency while preserving, even improving model capacity.

(3) PWC-style Decoder

The decoder also adopts the PWC structure, and follows a coarse-to-fine refinement structure (Sun et al., 2018). The module is implemented with a flow prediction head, a refinement module and a context network (dilated convolution). This module ends with the prediction:

$$\hat{F} = \mathcal{D}(z_t)$$

which is the final flow estimation.

(4) Self-Supervised Learning Implementation

In addition to the supervised flow estimation, our framework follows the self-supervised learning design proposed in Milestone 1. For those who don't have the ground truth flow to do supervised training in the KITTI dataset, we implement an image reconstruction-based strategy, where the predicted flow is used to warp one image to another:

$$\widehat{I_t} = \mathcal{W}(I_{t+1}, F)$$

where $\mathcal{W}(\cdot)$ denotes a warping function. The reconstructed image $\hat{I}_t$ is then compared with the original image I_t to compute a photometric loss:

$$\mathcal{L}_{self} = \left\| I_t - \widehat{I_t} \right\|_1$$

This mechanism allows the model to learn motion representations without relying solely on ground truth labels, which is particularly important given the limited supervision in multi-frame sequences. At the current stage, this implementation remains consistent with our Milestone 1 design and serves as a key component of the training pipeline.

1.2 Technical Adjustments from Stage 1

Compared to the original proposal, the following adjustments were made:

(1) Sobel Channel Augmentation

To address weak edge modeling, we introduced an optional Sobel-based channel augmentation in the Pairwise Flow Encoder. The motivation of this design is to provide the model with more explicit edge-related structural information at the input level, so that motion boundaries may be more easily distinguished during feature extraction. In this sense, the added Sobel channel serves as a lightweight physics-inspired or structure-aware

prior, which matches our intention to overcome the gap between interpretability and expressiveness.

However, it is also worth noting that this idea is not commonly adopted in modern state-of-the-art optical flow models. A possible reason is that as a strong feature extractor, convolutional neural networks are already capable of learning edge-sensitive filters automatically during training, making additional explicit gradient augmentation redundant. Therefore, although this modification is intuitively reasonable, its actual effectiveness in our framework may be trivial. At the current stage, we keep this Sobel channel augmentation as an optional design choice. Its usefulness will be further evaluated in the next milestone through additional experiments and comparison with the baseline encoder setting.

(2) Loss Function Improvement: Edge-Aware Smoothness

During initial experiments, we observed that the model performs poorly in edge regions, particularly around object boundaries such as vehicle boundaries. The predicted flow tends to be overly smooth, resulting in blurry boundaries for objects.

A key reason for this issue lies in the formulation of the original smoothness loss we designed in Milestone 1:

$$\mathcal{L}_{smooth} = |\partial_x F| + |\partial_y F|$$

Here, F denotes the predicted flow field, while $\partial_x F$and $\partial_y F$represent its spatial gradients. This loss penalizes differences in motion between neighboring pixels, encouraging the flow to vary smoothly across the image, which is motivated by the physical property that motion fields are typically smooth within objects in real-world scenes. However, this effectively imposes a global smoothness assumption, which forces even distinct objects to share similar motion, which is not true, especially at edge cases. In real-world scenarios, motion is often discontinuous at object boundaries.

To address this limitation, we adopt an edge-aware smoothness loss:

$$\mathcal{L}_{smooth} = e^{-|\partial_x I|} \cdot |\partial_x F| + e^{-|\partial_y I|} \cdot |\partial_y F|$$

In this formulation, I denotes the input image, and $|\ \partial I\ |$ measures the magnitude of image gradients, where larger values indicate stronger intensity changes, indicating a more likely correspondence to object boundaries. The exponential term $e^{-|\partial I|}$ acts as a weighting factor: it reduces the smoothness constraint in regions with strong image gradients, edge cases mostly, and preserves it in homogeneous regions. As a result, the model enforces smooth motion within objects while allowing discontinuities at boundaries.

This idea is widely adopted in self-supervised vision tasks. The paper, Unsupervised Monocular Depth Estimation with Left-Right Consistency (Godard et al., 2018), introduces this formulation for depth estimation, and UnFlow: Unsupervised Learning of Optical Flow (Meister et al., 2017), using a similar formula, demonstrates its effectiveness in optical flow estimation. With this support, we implemented this adaptation into our loss function to try to achieve a better performance, while its true effectiveness in our model requires further experimentation.

2. Demonstration of Initial Results

2.1 Experimental Setup and Comparison Design

At the current stage, we conduct preliminary experiments to evaluate the effectiveness of the proposed training strategy. Specifically, two versions of the model are implemented and compared under the same experimental setting.

Both models share an identical architecture consisting of:

- A pairwise flow encoder based on CNN and correlation layers,
- A joint visual–motion encoder with two convolutional branches,

- A PWC-style decoder for dense flow reconstruction,
- A self-supervised training framework incorporating photometric and temporal constraints.

Both models use basically the same structure for total loss functions, same as the design in Milestone 1, as follows:

$$\mathcal{L} = \mathcal{L}_{flow} + \lambda_{self}\mathcal{L}_{self} + \lambda_{temp}\mathcal{L}_{temp} + \lambda_{smooth}\mathcal{L}_{smooth}$$

where each component plays a distinct role in training.

The supervised flow loss $\mathcal{L}_{\text{flow}}$ is defined as the endpoint error (EPE) between the predicted flow and the ground truth: $\mathcal{L}_{\text{flow}} = \|F - F^{\text{gt}}\|_2$, which served as the dominant loss to ensure the accuracy of the predicted motion. The self-supervised photometric loss $\mathcal{L}_{\text{self}}$ is based on image reconstruction as introduced before: $\mathcal{L}_{\text{self}} = \left\|I_t - \widehat{I_t}\right\|_1$, which allows self-supervised learning implementation by encouraging consistency between the original image and the reconstructed image using the predicted flow. The temporal loss $\mathcal{L}_{\text{temp}}$ enforces consistency of motion across adjacent frames: $\mathcal{L}_{\text{temp}} = \|F_{t+1} - F_t\|_1$, with the intention of reflecting the physical property that objects tend to move consistently over time. The only difference between the two experimental settings lies in the formulation of the smoothness regularization term. The first model uses the original smoothness loss, while the second replaces it with the edge-aware smoothness loss that preserves motion discontinuities.

The weighting parameters are set as:

$$\lambda_{\text{self}} = 0.2, \;\; \lambda_{\text{temp}} = 0.1, \;\; \lambda_{\text{smooth}} = 0.01$$

All experiments are trained for 200 epochs on the DKU Computing Cluster using an NVIDIA Titan GPU, ensuring a consistent computational environment for fair comparison.

2.2 Initial Result Analysis

To analyze the convergence behavior of the models, we report the loss values from the final five epochs of training. Also, we visualize a few samples to analyze the initial performance more directly.

(1) Original Smoothness Loss:

After 200 epochs of training, the loss verbose of the last 5 epochs' performance is shown as follows:

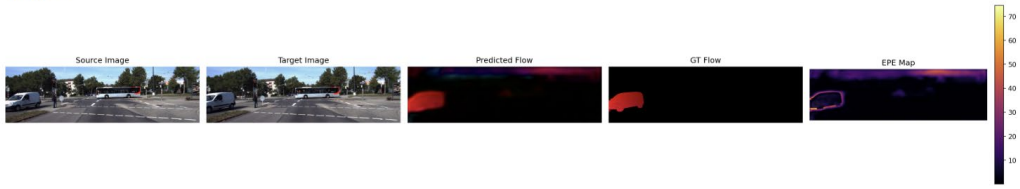
Epoch	**Total Loss**	$\mathcal{L}_{flow}$	$\mathcal{L}_{self}$	$\mathcal{L}_{temp}$	$\mathcal{L}_{smooth}$
196	4.4282	4.0339	**0.3876**	3.1321	0.3579
197	4.5403	4.1461	0.3885	3.1298	**0.3549**
198	4.3482	3.9568	0.3886	**3.1007**	0.3552
199	4.5114	4.1139	0.3878	3.1637	0.3568
200	**4.2671**	**3.8673**	0.3879	3.1859	0.3619
Total Loss $\mathcal{L} = \mathcal{L}_{flow} + 0.2\,\mathcal{L}_{self} + 0.1\,\mathcal{L}_{temp} + 0.01\,\mathcal{L}_{smooth}$					

Table 1: Last 5 of 200 epochs loss using smoothness loss function

Three visualization samples are also shown in Figure 2. The figures include two original images (Source – Target), model prediction of the optical flow between the two images, Ground truth of the flow provided by the KITTI dataset, and the EPE map, respectively. Note that since our dataset contains intrinsic pixel validity information, the EPE map shown includes the raw EPE logits. When applying only valid pixels to error measurements, the “blacked out” region would significantly increase.

- sample1:

- sample2:



- sample3:

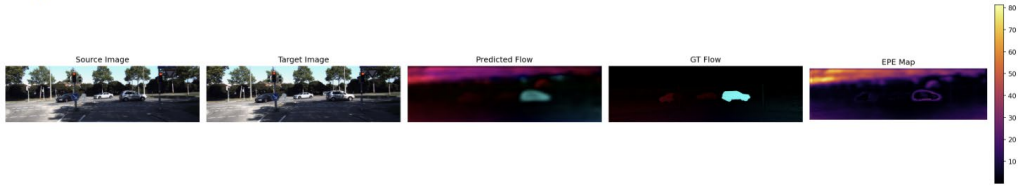


Figure 2: Samples of model training using the original loss function for 200 epochs

(2) Edge-Aware Smoothness Loss

Slight changes can be observed compared to the previous case. However, this can be attributed to the fact that the only difference between the two setups is the change from regularizing smoothness terms to edge awareness. Below reports the last 5 epochs of a 200-epoch training experiment:

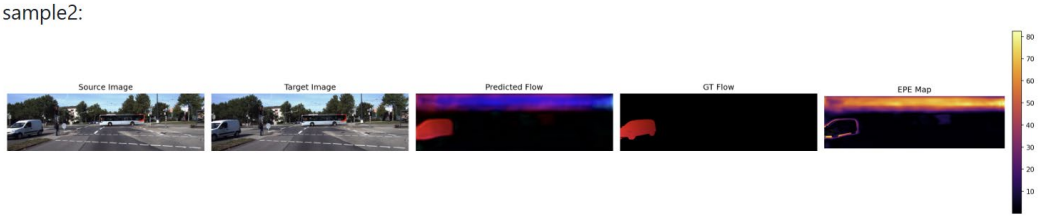
Epoch	Total Loss	$\mathcal{L}_{flow}$	$\mathcal{L}_{self}$	$\mathcal{L}_{temp}$	$\mathcal{L}_{smooth}$
196	4.1263	3.7433	0.3858	3.0159	0.4263
197	4.1415	3.7533	0.3845	3.0696	0.433
198	3.8996	**3.5234**	**0.3828**	2.9532	0.4312
199	**3.8964**	3.5269	0.3844	**2.8834**	**0.4255**
200	3.9639	3.5899	0.3856	2.9263	0.4298
Total Loss $\mathcal{L} = \mathcal{L}_{flow} + 0.2\,\mathcal{L}_{self} + 0.1\,\mathcal{L}_{temp} + 0.01\,\mathcal{L}_{smooth}$					

Table 2: Last 5 of 200 epochs loss using edge awareness loss function

Besides, three visual examples, using the exact same original images as the previous one, are used to compare the performance to check the effectiveness of the change of loss functions, as shown in Figure 3.

- sample1:

- sample2:



- sample3:

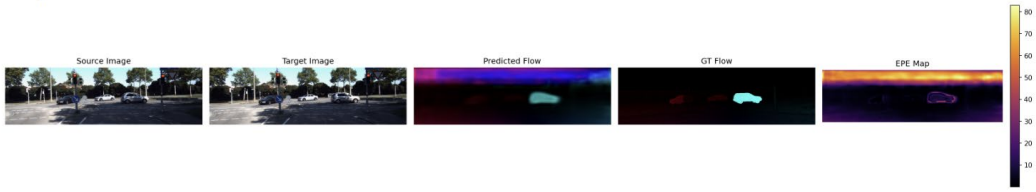


Figure 3: Samples of model training using the edge awareness loss function for 200 epochs

We also demonstrate our results on self-supervised short-horizon flow estimation. Following the same configuration, we are able to retrieve the $t-1$ estimated flows given the t input images for $t=4$ in the example shown in Figure 4. It is visible that the model now successfully identifies the major objects in motion, while specific edges might still underperform than expected. Additionally, estimation anomalies in the upper region of the image consistently present, posing another issue in proper regularization facing the existence of invalid pixels.

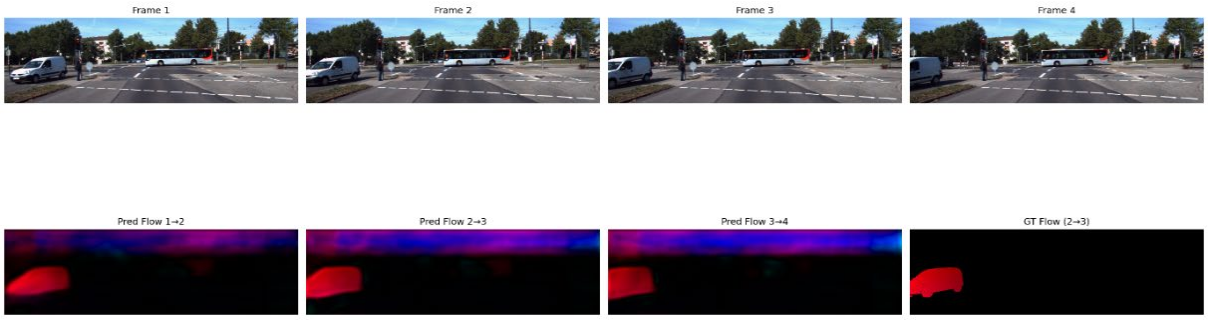


Figure 4: Example self-supervised short-horizon flow estimation with the same model configuration

2.3 Discussion

From both qualitative and quantitative results, several observations can be made.

First, the model trained with edge-aware smoothness loss achieves consistently lower total loss and flow loss compared to the original formulation. This suggests that incorporating image gradients into the smoothness constraint helps guide the model toward more accurate motion estimation. From the qualitative results, the edge-aware version appears to produce sharper motion boundaries, particularly around objects with clear edges such as vehicles. This aligns with the design intention of the edge-aware smoothness loss, which reduces over-smoothing across discontinuities. However, CNN-based feature reconstructors (i.e. the decoder) estimates the flows by up sampling from the latent space. Thus, they tend to produce overly smooth boundaries, and consequently led to higher smoothness loss values compared to a vanilla smoothness regularizer. This leaves space for future improvement.

Second, the self-supervised photometric loss remains relatively stable across both settings, indicating that the reconstruction objective is not significantly affected by the change in smoothness formulation. In contrast, the temporal loss remains relatively high in both cases, implying that temporal consistency is still challenging to enforce at the current stage, which we believe would be better after the operator is introduced.

It is important to note that the current pipeline is not yet fully complete, as the operator-based temporal module has not been integrated. Meanwhile, both models have not yet reached full

convergence at the 200-epoch stage, and the observed improvements should be interpreted with caution. Overall, while the edge-aware smoothness loss demonstrates promising improvements in handling edge cases, further experiments and architectural enhancements are required to validate its effectiveness in a fully developed framework.

3. Plan for the Next Two Weeks

We are currently in progress to implement the U-shape Neural Operator (UNO) module to enable our pipeline to learn physics-aware flow estimation (Wen et al., 2022). The next two weeks will focus on the implementation and fine-tuning of this module, which includes experiments on the following variations:

(1) Standalone UNO Implementation (Post-Fusion Module)

This is the original design of our framework. In this setup, UNO is applied as a post-processing operator on the fused spatiotemporal latent representation. All visual and motion features are first extracted and combined using the baseline pipeline, after which UNO refines the aggregated feature map. This design treats UNO as a global context enhancer, improving spatial coherence and long-range dependencies without altering earlier feature extraction stages. However, its influence is limited to late-stage refinement, as it operates on already highly processed representations in small shapes.

(2) UNO-Enhanced Spatiotemporal Fusion Module

This setup is similar to (1), but UNO is integrated directly within the spatiotemporal fusion process, replacing or augmenting conventional fusion operations. Instead of refining fused features, UNO actively governs how visual and motion information are combined, enabling structured interaction across spatial and temporal dimensions. This allows the operator to shape intermediate representations with global consistency, improving feature alignment and information flow. Compared to

standalone usage in (1), this design gives UNO a more central role in organizing multi-modal feature integration.

(3) UNO-Driven Image–Flow Coupling Module

In this formulation, UNO is embedded within the motion representation pipeline, operating on pairwise image-flow features after down sampling but before fusion with visual context. By acting directly on motion-related representations, UNO serves as a structural operator that models spatial dependencies and temporal evolution in the flow field. This design aligns most closely with the intended physics-inspired role of neural operators, as it shapes motion features at their source rather than refining them post hoc, leading to more principled and interpretable improvements. Nevertheless, this setup creates the most discrepancy compared to our initial design.

Meanwhile, we notice that as the KITTI dataset does not label all of its raw image pixels as valid, certain regions are consistently incomparable with the ground truth during training. Specifically, in downstream regression results, noisy background flow predicted in the sky is frequently observed, obscuring model performance evaluation. In this case, since the sky is relatively static compared to the egoistic motion captured by the camera, it is conventionally labeled as invalid in the training data, as well as in loss functions. Thus, the certain degree of data is free from regularization of loss functions, and tends to behave drastically. Even though the background noise shows the correct trend, we still intend to cope with this issue by introducing a masked end-point error (EPE) loss, along with modifying the current smoothness and temporal losses. By imposing stronger constraints on the data, the model should be able to perform better around invalid pixels.

To demonstrate the practical value of our framework, we plan to further propose incorporating a lightweight yet informative downstream module. This module performs automated bounding box generation to localize regions of motion, explicitly annotating

moving objects and optionally providing a coarse, low-level categorization of the detected instances. Unlike conventional object recognition pipelines, the objective is not fine-grained segmentation and semantic classification, but rather to illustrate that our model produces clear, stable, and physically meaningful motion estimations. These results serve as a reliable foundation for downstream perception tasks and highlight the potential of our approach for real-world applications such as autonomous driving.

References

Sun, D., Yang, X., Liu, M.-Y., & Kautz, J. (2018). PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8934–8943. https://doi.org/10.1109/CVPR.2018.00931

Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., & Benson, S. M. (2022). U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow. *Advances in Water Resources*, *163*, 104180. https://doi.org/10.1016/j.advwatres.2022.104180

Zhang, J., Chen, Y., & Huang, X. (2009). Edge detection of images based on improved Sobel operator and genetic algorithms. *2009 International Conference on Image Analysis and Signal Processing*, 31–35. https://doi.org/10.1109/IASP.2009.5054605